



**UNIVERSIDAD DE LA
CUENCA DEL PLATA**

ACTIVIDAD EVALUATIVA 2

CARRERA: Ingeniería en Sistemas de Información

MATERIA: Sistemas Operativos

PROFESOR: Ing. Walter Esteban Vasquez

ESTUDIANTE: Azcona Valentina

FECHA: 05/10/2026

Introducción

El presente informe documenta el desarrollo de un simulador discreto de gestión de procesos, memoria y CPU. El proyecto fue implementado como una biblioteca de clases en TypeScript y validado mediante pruebas automatizadas con Vitest. La simulación no depende del reloj real ni del sistema operativo anfitrión: cada llamada a `avanzarTick()` representa una unidad de tiempo de simulación.

El problema modelado corresponde a un laboratorio universitario que necesita analizar cómo diferentes procesos comparten una memoria limitada y una única CPU, incluyendo admisión, espera por memoria, planificación Round Robin, bloqueo por Entrada/Salida, liberación de recursos y medición de fragmentación.

Objetivos

Objetivo general: Implementar y verificar un simulador determinista que represente la administración de procesos, memoria y CPU mediante conceptos propios de Sistemas Operativos.

Objetivos específicos:

- Representar los estados de los procesos y sus transiciones válidas.
- Administrar memoria mediante asignación contigua y políticas First-Fit, Best-Fit y Worst-Fit.
- Implementar liberación de memoria y coalescencia automática de bloques libres adyacentes.
- Planificar la CPU con Round Robin y un quantum configurable.
- Simular eventos deterministas de Entrada/Salida.
- Calcular métricas de CPU y memoria, incluyendo fragmentación externa.
- Validar el comportamiento mediante tests reproducibles y cobertura de código superior al 90%.

Descripción general de la solución

La solución se organiza en componentes con responsabilidades diferenciadas. Proceso representa el estado y los contadores de cada proceso; BloqueMemoria representa una partición contigua; Memoria administra el mapa de memoria y las políticas de asignación; Planificador administra la CPU y la cola FIFO de Listos; Simulacion coordina las fases de cada tick y expone consultas del estado y las métricas.

La configuración permite definir memoria total, quantum y política de asignación. La simulación inicia en tick 0, con CPU libre, colas vacías y un único bloque libre que ocupa toda la memoria.

Diseño de la simulación

Cada tick se ejecuta de forma determinista y en un orden fijo. Esto permite que los escenarios puedan repetirse exactamente y que cada resultado sea verificable por pruebas.

- Admisión de procesos e intento de asignación de memoria.
- Actualización de procesos bloqueados por E/S.
- Despacho y ejecución de CPU mediante Round Robin.
- Actualización del reloj y de las métricas.

Como máximo un proceso consume una unidad de CPU por tick. Si un proceso finaliza durante la fase de CPU, su memoria se libera en ese mismo tick, pero los procesos que estaban esperando memoria vuelven a intentar ingresar recién en la fase de admisión del tick siguiente.

Estructuras de datos utilizadas

- Arreglos/listas para representar las colas de procesos Nuevos, Esperando Memoria, Listos, Bloqueados y Terminados.
- Una referencia al PID actualmente ejecutado por la CPU.
- Una lista ordenada de bloques de memoria contigua.
- Objetos Proceso para almacenar CPU restante, estado, quantum consumido y temporizador de bloqueo.
- Interfaces y objetos de estrategia para seleccionar First-Fit, Best-Fit o Worst-Fit.

Descripción de los estados de los procesos

Estado	Descripción
Nuevo	Proceso recién registrado, todavía no admitido.
Esperando Memoria	Proceso que no puede ingresar porque no existe un bloque contiguo suficiente.
Listo	Proceso con memoria asignada que espera turno de CPU.
Ejecutando	Proceso actualmente atendido por la CPU.
Bloqueado	Proceso suspendido temporalmente por un evento de Entrada/Salida; conserva su memoria.
Terminado	Proceso que consumió toda su CPU requerida y liberó su memoria.

Explicación del algoritmo implementado

La planificación de CPU utiliza Round Robin. Los procesos Listos se mantienen en una cola FIFO. Cuando la CPU queda libre, se despacha el primer proceso de la cola. Cada tick ejecutado reduce la CPU restante en una unidad y aumenta el quantum consumido.

Si el proceso termina, pasa a Terminado y su memoria se libera. Si se agota el quantum y existen otros procesos Listos, el proceso vuelve al final de la cola y se registra un cambio de contexto. Si no existen otros Listos, el quantum se renueva sin cambio de contexto. El bloqueo por E/S tiene prioridad sobre la rotación por quantum.

Para memoria se implementan tres políticas: First-Fit, Best-Fit y Worst-Fit. Todas trabajan sobre bloques contiguos ordenados por dirección y utilizan la misma lógica de partición y liberación.

Diagrama de transición de estados

El diagrama representa el ciclo de vida de un proceso dentro del simulador y muestra las transiciones permitidas entre los estados definidos en la clase Proceso.

El proceso comienza en el estado Nuevo cuando es registrado mediante registrarProceso(). Al inicio de un tick, durante la fase de admisión, pasa a Esperando Memoria, donde intenta obtener un bloque contiguo de memoria suficiente.

Si la asignación de memoria falla porque no existe un hueco adecuado, el proceso permanece en Esperando Memoria y vuelve a intentarlo en ticks posteriores. Cuando la asignación tiene éxito, el proceso pasa a Listo y se incorpora a la cola del planificador.

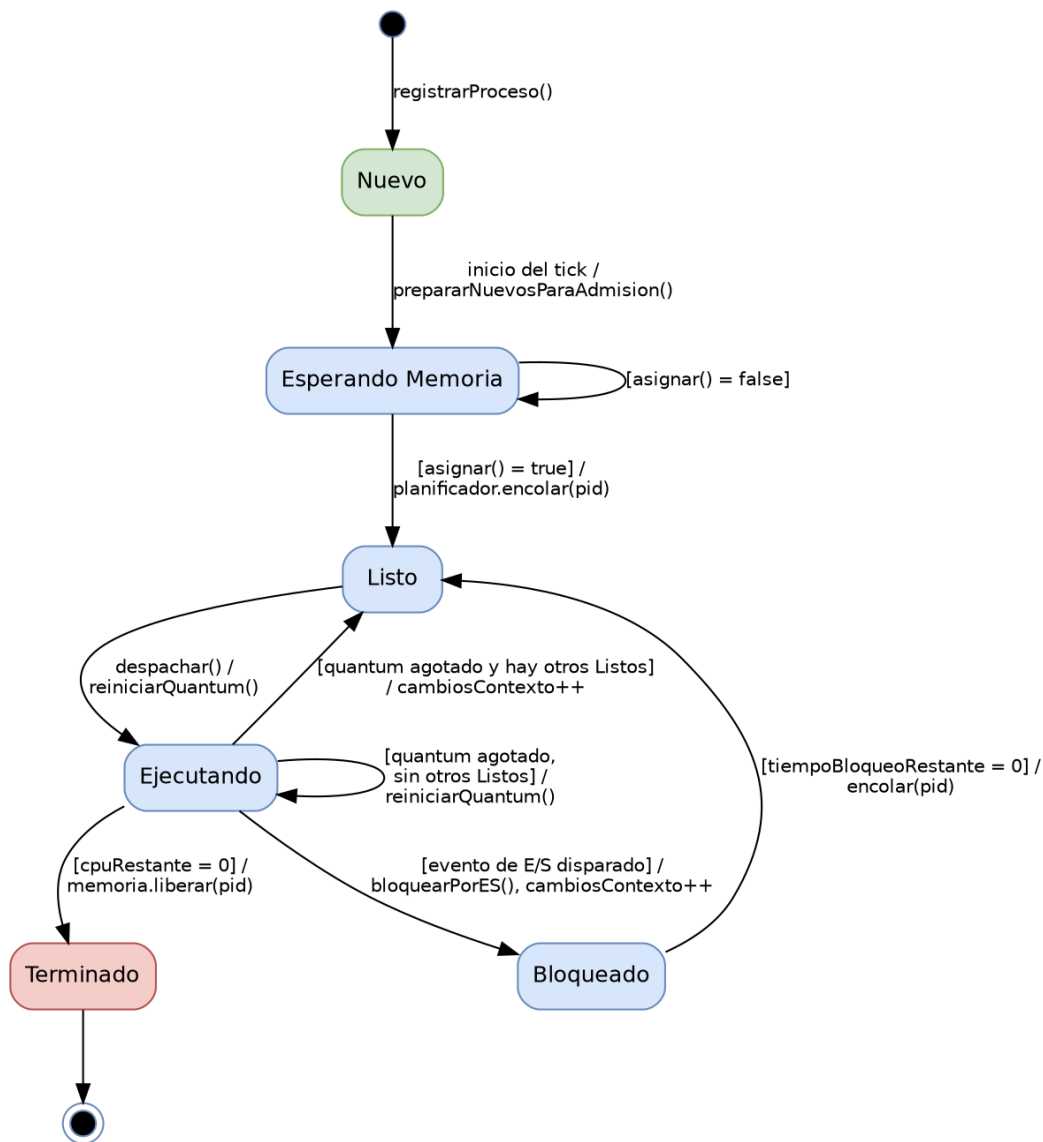
Desde Listo, el proceso puede pasar a Ejecutando cuando el planificador lo despacha a la CPU. Al comenzar su ejecución se reinicia el contador de quantum correspondiente a su turno.

Mientras se encuentra en Ejecutando, pueden ocurrir distintas situaciones. Si se agota el quantum y existen otros procesos en estado Listo, el proceso vuelve al final de la cola de Listos y se registra un cambio de contexto. Si el quantum se agota pero no existen otros procesos esperando CPU, el proceso permanece en ejecución y simplemente se reinicia su quantum, sin contabilizar un cambio de contexto.

También desde Ejecutando, si se dispara un evento de Entrada/Salida, el proceso pasa al estado Bloqueado. En este caso libera la CPU, conserva la memoria asignada y se incrementa el contador de cambios de contexto. Mientras permanece bloqueado, su temporizador de E/S se reduce en los ticks siguientes. Cuando dicho temporizador llega a cero, el proceso retorna al estado Listo y vuelve a incorporarse a la cola del planificador.

Finalmente, cuando la CPU restante del proceso llega a cero, este pasa al estado Terminado. En ese momento se libera la memoria que tenía asignada y se ejecuta la coalescencia de bloques libres adyacentes. El estado Terminado es definitivo, por lo que el proceso no vuelve a ninguna cola ni puede ser ejecutado nuevamente.

El diagrama también muestra las prioridades definidas por el simulador: la finalización tiene prioridad sobre otros eventos, y el bloqueo por Entrada/Salida tiene prioridad sobre el agotamiento del quantum. De esta manera se mantiene un comportamiento determinista y coherente con las reglas establecidas para cada tick.



Mecanismo de coalescencia

La coalescencia se ejecuta automáticamente después de liberar un bloque. Su objetivo es fusionar bloques libres adyacentes para reducir la fragmentación externa. No mueve bloques ocupados, por lo que no constituye compactación.

Casos de prueba

Caso	Qué se verifica
Configuración y registro	Estado inicial correcto; parámetros inválidos; PID duplicados; procesos mayores que la memoria total.
Asignación y espera	Partición parcial, ajuste exacto, selección según política, espera y posterior admisión.
Fragmentación	La suma libre alcanza pero ningún hueco contiguo es suficiente; la asignación debe fallar sin alterar bloques.

Coalescencia	Fusión con vecino izquierdo, derecho y ambos; al liberar todo queda un único bloque libre.
Round Robin	Con Q=2 y procesos CPU 3 y 2, verificar secuencia P1, P1, P2, P2, P1.
Quantum	Un único proceso renueva quantum sin cambio de contexto.
Finalización	Finalizar en el límite de quantum no reencola el proceso.
Entrada/Salida	Bloqueo, conservación de memoria, no consumo de CPU y retorno a Listos al vencer el temporizador.
Métricas	Tick 0, memoria llena, utilización de CPU y fragmentación externa.
Invariantes	Sin procesos duplicados, sin solapamientos y máximo un proceso Ejecutando.

Resultados obtenidos

La versión final verificada compila sin errores de TypeScript y todas las pruebas automatizadas pasan correctamente.

Indicador	Resultado
Suites de prueba	11
Tests aprobados	76 / 76
Tests fallidos	0
Typecheck	Sin errores
Statements	94.54%
Branches	91.46%
Functions	97.64%
Lines	94.54%

La cobertura global de líneas es 94.54%, por lo que supera el 90% requerido.

Análisis comparativo de First-Fit, Best-Fit y Worst-Fit

Para comparar las tres políticas se utilizó el mismo mapa de memoria con huecos de diferente tamaño y una nueva solicitud que podía entrar en más de uno. En el escenario de prueba quedaron huecos libres de 200, 300 y 150 unidades, ubicados en distintas direcciones. Luego se solicitó un bloque de 140 unidades.

Política	Bloque seleccionado	Interpretación
First-Fit	Hueco de 200 (primera dirección disponible)	Prioriza rapidez de búsqueda y la primera coincidencia suficiente.
Best-Fit	Hueco de 150	Minimiza el sobrante inmediato; deja un residuo pequeño.
Worst-Fit	Hueco de 300	Utiliza el mayor hueco disponible y conserva un residuo relativamente grande.

La comparación muestra que una misma solicitud puede producir distribuciones residuales diferentes. First-Fit depende fuertemente del orden de los bloques; Best-Fit intenta reducir el desperdicio inmediato, aunque puede generar huecos pequeños; Worst-Fit utiliza el hueco mayor y puede conservar espacios residuales más grandes. Por lo tanto, ninguna política es universalmente mejor: su comportamiento depende de la secuencia de asignaciones y liberaciones.

Capturas de pantalla de la aplicación en funcionamiento

Como la solución entregada es una biblioteca sin interfaz gráfica ni menú de consola, las evidencias de funcionamiento corresponden a la ejecución de los tests y al reporte de cobertura.

[Insertar captura de npm test mostrando 76/76 tests aprobados]

[Insertar captura de npm run coverage mostrando los porcentajes de cobertura]

[Opcional: insertar captura del mapa/estado obtenido desde un test representativo]

Análisis de situaciones de fragmentación externa

La fragmentación externa ocurre cuando la memoria libre está dividida en varios huecos no contiguos. En ese caso, la suma total de memoria libre puede ser suficiente, pero una solicitud puede fallar porque requiere un único bloque contiguo.

La métrica utilizada es: Fragmentación externa = $100 \times (1 - \text{mayor bloque libre} / \text{memoria libre total})$. Si la memoria libre total es 0, la fragmentación se define como 0%.

En el caso mínimo de prueba existen huecos libres de 100 KB y 300 KB. La memoria libre total es 400 KB y el mayor hueco es 300 KB, por lo que la fragmentación es $100 \times (1 - 300/400) = 25\%$.

La coalescencia reduce la fragmentación cuando los huecos libres quedan adyacentes, ya que los fusiona. En cambio, si existen bloques ocupados entre los huecos, la coalescencia no puede unirlos. El simulador no implementa compactación, por lo que los bloques ocupados no se desplazan.

Conclusión

El simulador desarrollado permite estudiar de forma controlada y reproducible la interacción entre procesos, memoria y CPU. La implementación de Round Robin, los estados de procesos, la Entrada/Salida determinista, la asignación contigua, la coalescencia y las métricas permiten observar fenómenos reales de los Sistemas Operativos sin depender del tiempo real.

La comparación entre First-Fit, Best-Fit y Worst-Fit evidencia que la política seleccionada modifica la distribución de la memoria libre y, por lo tanto, puede influir en la fragmentación externa. Las pruebas automatizadas y la cobertura obtenida proporcionan evidencia de que los casos normales, límites y errores principales fueron considerados.